```
"""
RIPS-frontier: Reachability via Incremental Pruning of Subtrees
Directed graph on a fixed vertex set of size V, source s, hashable
labels.
Self-loops and duplicate arcs are ignored. Work units: one vertex
reset or
one edge examination. Lemma 3 bounds the work of each update by V +
2m.
"""

from collections import defaultdict, deque

INF = float("inf")


class RipsFrontier:
    def __init__(
        self,
        s,
        n_vertices,
        c0=0.5,
        ca=0.5,
        cf=1.0,
        cb=0.2,
        cq=0.1,
        delta=0.2,
        alpha=0.05,
        window=50,
    ):
        self.s = s
        self.V = n_vertices
        self.c0, self.ca, self.cf, self.cb, self.cq = c0, ca, cf, cb,
cq
        self.delta, self.alpha, self.window = delta, alpha, window

        self.out_edges = defaultdict(set)
        self.in_edges = defaultdict(set)
        self.m = 0

        self.mode = "frontier"
        self.mu = 0.0
        self.p_hit = 0.0
        self.A_w = 0
        self.Q_w = 0
        self.ops = 0
        self.transitions = 0

        self.last_work = 0
```

```
    self.total_work = 0
    self.last_query_work = 0
    self.rebuild_work = 0

    self.rebuild()

def insert(self, u, v):
    changed = self._insert(u, v)
    if changed:
        self.A_w += 1
        self._tick()
    return changed

def delete(self, u, v):
    changed = self._delete(u, v)
    if changed:
        self.A_w += 1
        self._tick()
    return changed

def query(self, x):
    self.last_query_work = 0
    if self.mode == "frontier":
        answer = x in self.reachable
    else:
        answer = x in self._bfs()
    self.Q_w += 1
    self._tick()
    return answer

def rebuild(self):
    self.reachable = {self.s}
    self.parent = {}
    self.children = defaultdict(set)
    work = 1
    dq = deque([self.s])

    while dq:
        y = dq.popleft()
        for z in self.out_edges[y]:
            work += 1
            if z not in self.reachable:
                self.reachable.add(z)
                work += 1
                self.parent[z] = y
                self.children[y].add(z)
                dq.append(z)
    self.rebuild_work += work
```

```
    def _insert(self, u, v):
        self.last_work = 0
        if u == v or v in self.out_edges[u]:
            return False

        self.out_edges[u].add(v)
        self.in_edges[v].add(u)
        self.m += 1

        if self.mode == "batch":
            return True

        work = 0
        if u in self.reachable and v not in self.reachable:
            self.reachable.add(v)
            self.parent[v] = u
            self.children[u].add(v)
            dq = deque([v])

            while dq:
                y = dq.popleft()
                for z in self.out_edges[y]:
                    work += 1
                    if z not in self.reachable:
                        self.reachable.add(z)
                        self.parent[z] = y
                        self.children[y].add(z)
                        dq.append(z)
        self._record(work)
        return True

    def _delete(self, u, v):
        self.last_work = 0
        if u == v or v not in self.out_edges[u]:
            return False

        self.out_edges[u].discard(v)
        self.in_edges[v].discard(u)
        self.m -= 1

        if self.mode == "batch":
            return True

        hit = self.parent.get(v) == u
        self.p_hit = (1 - self.alpha) * self.p_hit + self.alpha *
float(hit)
        work = 0
```

```
    if hit:
        self.children[u].discard(v)
        T, stack = set(), [v]
        while stack:
            x = stack.pop()
            if x not in T:
                T.add(x)
                stack.extend(self.children[x])

        work += len(T)
        for x in T:
            self.reachable.discard(x)
            self.parent.pop(x, None)
            self.children[x] = set()

        frontier = deque()
        for x in T:
            for w in self.in_edges[x]:
                work += 1
                if w in self.reachable:
                    self.reachable.add(x)
                    self.parent[x] = w
                    self.children[w].add(x)
                    frontier.append(x)
                    break

        while frontier:
            y = frontier.popleft()
            for z in self.out_edges[y]:
                work += 1
                if z in T and z not in self.reachable:
                    self.reachable.add(z)
                    self.parent[z] = y
                    self.children[y].add(z)
                    frontier.append(z)

    self._record(work)
    return True

def _record(self, work):
    self.last_work = work
    self.total_work += work
    self.mu = (1 - self.alpha) * self.mu + self.alpha * work

def _bfs(self):
    seen = {self.s}
    work = 1
```

```
        dq = deque([self.s])
        while dq:
            y = dq.popleft()
            for z in self.out_edges[y]:
                work += 1
                if z not in seen:
                    seen.add(z)
                    work += 1
                    dq.append(z)
        self.last_query_work = work
        return seen

    def r_star(self):
        num = self.cb * (self.V + self.m) - self.cq
        den = self.cf * self.mu + self.c0 - self.ca
        if num <= 0:
            return 0.0
        if den <= 0:
            return INF
        return num / den

    def _tick(self):
        self.ops += 1
        if self.ops % self.window == 0:
            self._end_window()

    def _end_window(self):
        r = self.A_w / max(self.Q_w, 1)
        r_star_val = self.r_star()

        if self.mode == "frontier" and r > (1 + self.delta) *
r_star_val:
            self.mode = "batch"
            self.transitions += 1
        elif self.mode == "batch" and r < (1 - self.delta) *
r_star_val:
            self.mode = "frontier"
            self.rebuild()
            self.transitions += 1

        self.A_w = 0
        self.Q_w = 0
```